Explainable Deep Learning for IoT Sensor-Based Machine Failure Prediction in Predictive Maintenance

[Author 1 Name]
[Department name]
[Institution name]
[City, Country]
[email address or ORCID]

[Author 2 Name]
[Department name]
[Institution name]
[City, Country]
[email address or ORCID]

[Author 3 Name]
[Department name]
[Institution name]
[City, Country]
[email address or ORCID]

***Abstract*—Predictive maintenance relies on sensor data to anticipate machine failures before they occur, but deep learning models used for this task are often treated as black boxes, which limits operator trust. This paper presents an explainable failure-prediction pipeline evaluated on the AI4I 2020 predictive maintenance dataset of 10,000 machine records with a failure rate of only 3.39%. Three deep learning architectures, a multilayer perceptron (MLP), a long short-term memory network (LSTM) and a one-dimensional convolutional neural network (1D-CNN), are trained on the same stratified split with a class-weighted loss to counter the severe imbalance. On a held-out test set of 1,500 machines, the MLP achieves the best overall discrimination with a ROC-AUC of 0.976 and a recall of 90.2%, while the 1D-CNN reaches the highest recall of 94.1%. A threshold analysis shows that raising the MLP decision threshold to 0.93 increases the F1-score from 0.413 to 0.724. SHapley Additive exPlanations (SHAP) are used to produce global and per-prediction explanations, and a what-if analysis shows that the model's learned decision boundaries align with the physical failure mechanisms of tool wear and power overload. The model and its explanations are served through a REST API with a median explanation latency of 17.6 ms and an interactive web dashboard.**

Keywords—predictive maintenance, explainable artificial intelligence, SHAP, deep learning, Internet of Things, class imbalance, failure prediction

I. Introduction

Unplanned machine downtime is one of the largest avoidable costs in manufacturing. Maintenance strategies have evolved from reactive repair and fixed-interval preventive maintenance towards predictive maintenance (PdM), in which the condition of equipment is monitored continuously and maintenance is scheduled only when a failure is likely [1], [2]. The spread of low-cost Internet of Things (IoT) sensors has made it practical to record operating variables such as temperature, rotational speed, torque and tool wear at scale, and machine learning models trained on these signals are now widely studied for failure prediction [3], [4].

Deep neural networks can learn non-linear interactions between sensor variables without manual feature engineering [5], but two practical obstacles limit their adoption on the shop floor. First, failures are rare, so a model can reach high accuracy while missing most failures [6]. Second, a maintenance engineer is unlikely to act on an alarm that cannot be justified. Explainable artificial intelligence (XAI) addresses the second problem by attributing a model's output to its inputs [7]; SHapley Additive exPlanations (SHAP) [8] in particular provides explanations with a solid game-theoretic foundation that can be read both globally and for individual predictions.

This paper combines both concerns in a single, reproducible pipeline built on the AI4I 2020 predictive maintenance dataset [9], [10]. The main contributions are:

- A like-for-like comparison of an MLP, an LSTM and a 1D-CNN trained on the same stratified split with the same class-weighted loss, optimiser and early-stopping protocol.
- An evaluation that goes beyond accuracy, covering recall, precision-recall behaviour, decision-threshold trade-offs and detection rates for each of the five underlying failure modes.
- Global and local SHAP explanations of the best model, including a what-if analysis that relates the learned decision boundaries to the physical failure mechanisms of the dataset.
- A deployable system that serves predictions and explanations through a REST API and an interactive web dashboard.

The rest of the paper is organised as follows. Section II reviews related work, Section III describes the dataset and preprocessing, Section IV presents the models, the training protocol and the explanation method, Section V reports the results, Section VI discusses them and Section VII concludes.

II. Related Work

Surveys of PdM report a steady shift from physics-based and statistical models towards data-driven machine learning, with random forests, support vector machines and neural networks being the most frequently used methods [3], [4]. Deep learning has been applied to machine health monitoring using auto-encoders, convolutional networks and recurrent networks, mostly on high-frequency vibration or run-to-failure signals [5]. Convolutional networks with one-dimensional kernels are attractive for sensor data because they are compact and fast at inference [11], while LSTM networks [12] are the standard choice when long temporal dependencies must be modelled.

The AI4I 2020 dataset was introduced by Matzka [9] together with an explainable decision-tree-based approach, and has since become a common benchmark because it provides labelled failure modes alongside the overall failure flag. For explainability, model-agnostic methods such as LIME [13] and SHAP [8] are the most widely used [7]. SHAP unifies several earlier attribution methods under the Shapley value from cooperative game theory [14] and

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

satisfies local accuracy, missingness and consistency. Compared with prior work, this study evaluates three deep architectures under identical conditions, analyses performance per failure mode and links the explanations back to the physical failure rules.

III. Dataset and Preprocessing

A. The AI4I 2020 Dataset

The AI4I 2020 predictive maintenance dataset [10] contains 10,000 records of a milling process with 14 columns. Each record describes a product of quality variant L (low, 60%), M (medium, 30%) or H (high, 10%) and five operating measurements: air temperature, process temperature, rotational speed, torque and tool wear. The binary target *Machine failure* is positive for 339 records (3.39%). Five failure-mode flags explain why a failure occurred: tool wear failure (TWF, 46 records), heat dissipation failure (HDF, 115), power failure (PWF, 95), overstrain failure (OSF, 98) and random failure (RNF, 19); one record can have more than one mode. The data contain no missing values and no duplicate rows. Table I summarises the numerical features and shows that failed machines run, on average, with higher torque and considerably more tool wear.

TABLE I. Numerical Sensor Features of the AI4I 2020 Dataset

Feature	Range	Mean (no failure)	Mean (failure)
Air temperature [K]	295.3–304.5	299.97	300.89
Process temp. [K]	305.7–313.8	310.00	310.29
Rot. speed [rpm]	1168–2886	1540.3	1496.5
Torque [Nm]	3.8–76.6	39.63	50.17
Tool wear [min]	0–253	106.7	143.8

Failure rates also differ by product variant: 3.92% for type L, 2.77% for type M and 2.09% for type H. Air and process temperature are strongly correlated (Pearson r = 0.876), as are rotational speed and torque (r = −0.875); all other pairwise correlations are below 0.03.

B. Feature Selection and Encoding

Six inputs are used: the product type and the five sensor measurements. The identifiers *UDI* and *Product ID* carry no physical information and are dropped. The five failure-mode flags are also excluded because they are only known after a failure and would leak the target. The five numerical features are standardised to zero mean and unit variance, and the product type is one-hot encoded, giving eight model inputs. The encoder and scaler are fitted on the training set only and stored with the models so that exactly the same transformation is applied at inference time.

C. Data Splitting

The data are split into training, validation and test sets in a 70/15/15 ratio using stratified sampling with a fixed random seed, so that every subset keeps the original failure rate. The training set has 7,000 records (237 failures), and the validation and test sets have 1,500 records each (51 failures each). The validation set is used only for early stopping, and all reported results are computed once on the untouched test set.

IV. Methodology

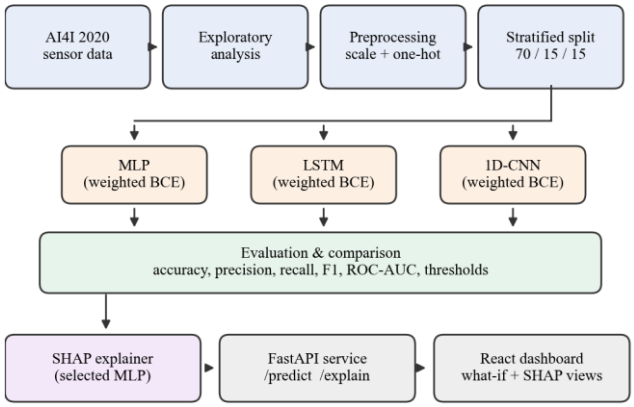


Fig. 1. Overview of the proposed explainable failure-prediction pipeline.

Fig. 1 summarises the pipeline. After preprocessing, three deep learning models are trained and compared, the best model is explained with SHAP, and the trained artifacts are served to a web dashboard through a REST API.

A. Model Architectures

MLP. The multilayer perceptron maps the eight inputs through two fully connected hidden layers of 64 and 32 ReLU units, with dropout [15] of 0.25 and 0.20, to a single output logit.

LSTM. The feature vector is presented to a two-layer LSTM [12] with 32 hidden units and inter-layer dropout of 0.2 as a sequence of eight steps with one value per step. The last hidden state feeds a 16-unit ReLU layer and an output logit.

1D-CNN. The feature vector is treated as a one-channel signal of length eight and processed by two convolutional layers (32 and 64 filters, kernel size 3, same padding), each followed by ReLU and batch normalisation [16]. Global average pooling and a 32-unit dense layer with dropout of 0.25 produce the output logit.

Because the AI4I records are independent snapshots rather than time series, the LSTM and 1D-CNN operate on a pseudo-sequence over features, not over time. Including them tests whether sequence-oriented inductive biases help on this kind of tabular sensor data. Table II lists the size of each model.

TABLE II. Model Architectures and Trainable Parameters

Model	Main layers	Parameters
MLP	Dense 64–32–1, dropout	2,689
LSTM	2×LSTM(32), dense 16–1	13,473
1D-CNN	Conv 32–64, GAP, dense 32–1	8,641

B. Class-Imbalance-Aware Training

With 237 failures among 7,000 training records, an unweighted classifier would be biased towards the majority class [6]. All three models are therefore trained with a positively weighted binary cross-entropy loss, where the positive-class weight equals the ratio of negative to positive training samples:

$$w_+ = \frac{N_-}{N_+} = \frac{6763}{237} \approx 28.5 \quad (1)$$

For a sample with label y_i and output logit z_i the loss is

$$\ell_i = -\left[w_+ y_i \log \sigma(z_i) + (1 - y_i) \log\left(1 - \sigma(z_i)\right)\right] \quad (2)$$

where σ is the logistic sigmoid, and ℓ_i is averaged over each mini-batch. All models use the Adam optimiser [17] with a learning rate of 0.001 and weight decay of 10^{-5}, a batch size of 64 and at most 100 epochs. Training stops early when the validation loss has not improved for 10 consecutive epochs, and the weights with the lowest validation loss are restored. Random seeds are fixed for reproducibility. The models are implemented in PyTorch [18] and the preprocessing and metrics use scikit-learn [19]. A failure is predicted when the sigmoid output is at least 0.5 unless stated otherwise.

C. Evaluation Metrics

Accuracy is reported for completeness but is misleading at a 3.4% failure rate. The primary metrics are therefore recall (the share of real failures that are detected), precision, F1-score, the area under the ROC curve (ROC-AUC) and the area under the precision-recall curve (PR-AUC), which is more informative than ROC-AUC for imbalanced data [20]. Recall is treated as the most important single metric because a missed failure usually costs far more than an unnecessary inspection.

D. Explainability with SHAP

SHAP explains a prediction f(x) by assigning each feature j a contribution φ_j equal to its Shapley value [8], [14], i.e., its average marginal contribution over all subsets S of the feature set F:

$$\varphi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!(|F|-|S|-1)!}{|F|!} \left[f_{S\cup\{j\}}(x_{S\cup\{j\}}) - f_S(x_S)\right] \quad (3)$$

The contributions are additive, so the base value φ_0 (the expected model output over a background set) plus the contributions of the M features recovers the prediction exactly:

$$f(x) = \varphi_0 + \sum_{j=1}^{M} \varphi_j \quad (4)$$

Here f is the failure probability produced by the MLP. Because the model has only eight inputs, the Shapley values are computed exactly rather than approximated, using a background set of 100 randomly sampled training records. Global importance is the mean absolute SHAP value over 100 test records, and local explanations are computed on demand for any input. Positive values push the prediction towards failure and negative values push it towards normal operation.

E. System Implementation

The trained MLP, the fitted preprocessor and the SHAP explainer are loaded once by a FastAPI service that exposes a /predict endpoint (failure probability and decision), an /explain endpoint (prediction with SHAP contributions sorted by magnitude) and a /sweep endpoint for what-if analysis. A React dashboard consumes these endpoints. It lets a user adjust the sensor readings with sliders and see the risk and its SHAP explanation update in real time, load real records from the dataset, compare the three models and move the decision threshold interactively.

V. Results

A. Classification Performance

Table III reports the test-set performance of the three models at the default threshold of 0.5. All models detect at least 88% of failures. The MLP achieves the highest accuracy, precision, F1-score, ROC-AUC and PR-AUC while being the smallest model, and the 1D-CNN achieves the highest recall at the cost of more false alarms. The LSTM is the weakest model on every metric except recall, where it is close to the MLP. The PR-AUC values make the differences clearer than ROC-AUC: 0.720 for the MLP against 0.599 for the 1D-CNN and 0.485 for the LSTM.

TABLE III. Test-Set Performance (Threshold 0.5, N = 1,500, 51 Failures)

Metric	MLP	LSTM	1D-CNN
Accuracy	**0.913**	0.903	0.890
Precision	**0.267**	0.245	0.229
Recall	0.902	0.882	**0.941**
Specificity	**0.913**	0.904	0.888
F1-score	**0.413**	0.383	0.368
ROC-AUC	**0.976**	0.959	0.965
PR-AUC	**0.720**	0.485	0.599

[a.] Best value per row in bold. PR-AUC is computed as average precision.

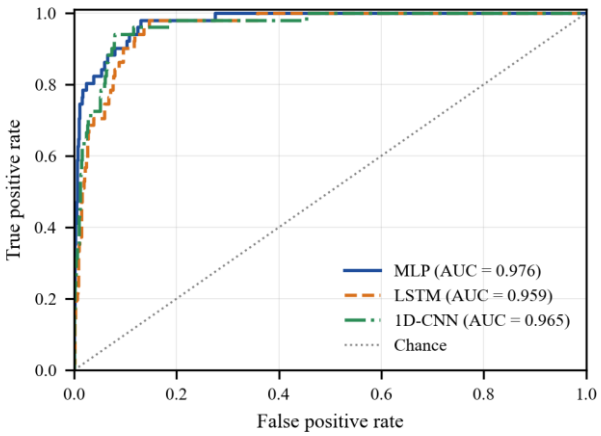


Fig. 2. ROC curves of the three models on the test set.

The ROC curves in Fig. 2 rise steeply for all models, and the confusion matrices in Fig. 3 show the practical trade-off. The MLP misses 5 of the 51 test failures and raises 126 false alarms among 1,449 healthy machines; the 1D-CNN misses only 3 failures but raises 162 false alarms. The models largely agree on which failures are hard: only 2 failures are missed by all three models, and a majority vote of the three models performs no better than the MLP alone (recall 0.902, F1-score 0.397).

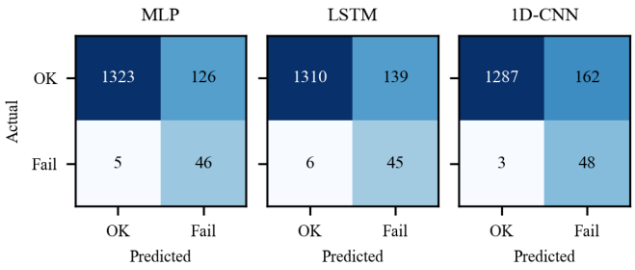


Fig. 3. Confusion matrices at a decision threshold of 0.5 (rows: actual class, columns: predicted class).

B. Decision-Threshold Analysis

The class-weighted loss deliberately shifts the models towards high recall, which is why precision is low at the default threshold. Because ROC-AUC is high, the threshold can be moved to suit the cost of each type of error. Table IV shows this for the MLP. Raising the threshold from 0.5 to 0.9 reduces false alarms from 126 to 24 while missing 6 more failures, and the F1-optimal threshold of 0.93 gives a precision of 0.704, a recall of 0.745 and an F1-score of 0.724. The corresponding best F1-scores are 0.604 for the 1D-CNN and 0.551 for the LSTM, both at a threshold of about 0.89, so the MLP remains the best model at every operating point.

TABLE IV. EFFECT OF THE DECISION THRESHOLD ON THE MLP

Thresh.	Precision	Recall	F1	FP	FN
0.30	0.190	0.980	0.318	213	1
0.50	0.267	0.902	0.413	126	5
0.70	0.378	0.824	0.519	69	9
0.80	0.466	0.804	0.590	47	10
0.90	0.625	0.784	0.696	24	11
0.93	0.704	0.745	0.724	16	13

[b] FP = false alarms, FN = missed failures on the test set. 0.93 is the F1-optimal threshold.

C. Detection by Failure Mode

Table V breaks down the detected test failures by their recorded failure mode. All models detect every power failure and every overstrain failure, and the 1D-CNN also detects all heat dissipation failures. Tool wear failures are the hardest to predict: each model detects only 5 of the 8 cases. Random failures, which occur with a small fixed probability regardless of operating conditions, cannot be predicted from the sensors, and only one of the two cases is flagged.

TABLE V. DETECTED TEST FAILURES BY FAILURE MODE

Mode	Count	MLP	LSTM	1D-CNN
TWF	8	5	5	5
HDF	18	16	16	18
PWF	13	13	12	13
OSF	15	15	15	15
RNF	2	1	1	1

[c] Counts overlap because a record can have more than one failure mode.

D. Global Explanations

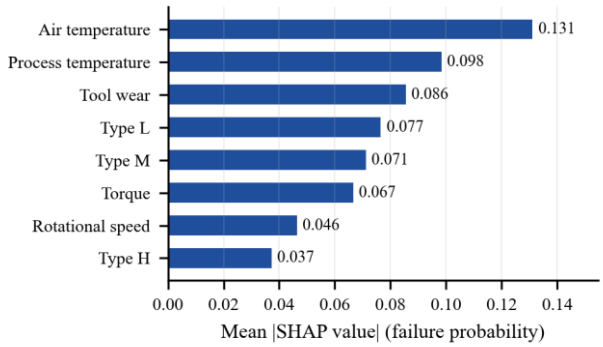


Fig. 4. Global SHAP feature importance of the MLP (mean absolute SHAP value over 100 test records).

Fig. 4 ranks the inputs by their mean absolute SHAP value. Air temperature (0.131) and process temperature (0.098) are the most influential sensors, followed by tool wear (0.086), torque (0.067) and rotational speed (0.046). The two temperatures matter mostly as a pair: heat dissipation failures occur when the gap between them becomes small at low speed [10], and a network can only model that gap through both inputs. The three one-hot product-type indicators together contribute 0.185, which is more than any single sensor and is consistent with the clearly different failure rates of the L, M and H variants.

E. Local Explanations and What-If Analysis

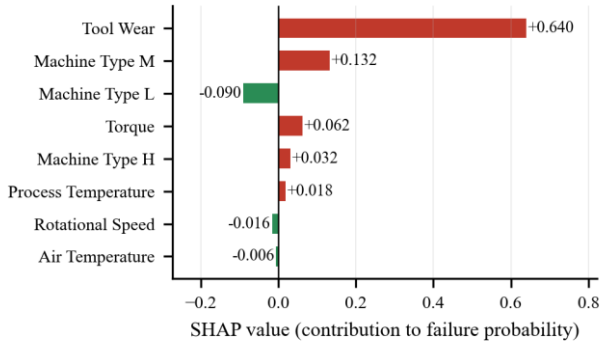


Fig. 5. Local SHAP explanation for a type L machine with 225 min of tool wear (failure probability 0.876).

Fig. 5 explains a single prediction for a type L machine running at 1,450 rpm and 48.1 Nm with 225 min of tool wear. The MLP predicts a failure probability of 0.876, and the explanation attributes most of this risk to tool wear (+0.640), with smaller contributions from torque (+0.062) and process temperature (+0.018). The product type is spread over three one-hot features with opposite signs (+0.132 for M, −0.090 for L and +0.032 for H); their sum of +0.074 is the net effect of the machine being type L. Explanations for one-hot encoded inputs should therefore be read as a group rather than one indicator at a time.

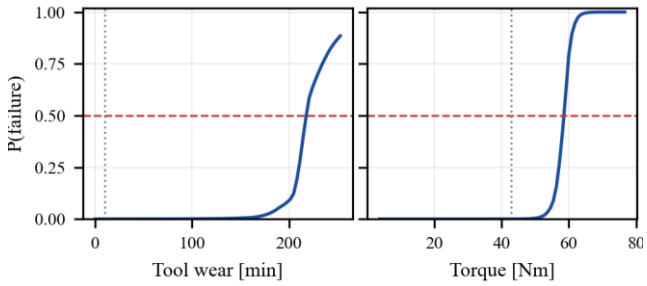


Fig. 6. What-if analysis for a healthy type M machine: failure probability as tool wear (left) or torque (right) varies while all other readings are fixed (dotted line: current value).

Fig. 6 varies one reading at a time for a healthy type M machine (298.5 K, 309.1 K, 1,408 rpm, 42.8 Nm, 10 min). The predicted risk crosses 0.5 at a tool wear of about 221 min, inside the 200–240 min window in which tool wear failures are generated in the dataset [10]. It also crosses 0.5 at a torque of about 59.1 Nm, which at 1,408 rpm corresponds to a mechanical power of about 8.7 kW, close to the 9 kW upper limit that defines power failures. The model has therefore learned decision boundaries that match the physical failure mechanisms, even though the failure-mode labels were never used in training.

F. *Inference Latency*

Served through the REST API on a laptop CPU, the median end-to-end response time over 20 requests is 13.2 ms for a prediction and 17.6 ms for a prediction with an exact SHAP explanation (95th percentile 14.9 ms and 18.3 ms). Explanations can therefore be produced for every prediction in real time without specialised hardware.

VI. Discussion

The results show that, for this snapshot-style tabular sensor data, the simplest architecture is also the best. The MLP outperforms the LSTM and 1D-CNN on every threshold-independent metric with five times fewer parameters than the LSTM. This agrees with the wider observation that sequence and convolutional inductive biases do not help when the feature order carries no temporal or spatial meaning [21]; for the LSTM and 1D-CNN the arbitrary order of the eight inputs becomes part of the model. These architectures would be expected to be more useful with genuine sensor time series.

The low precision at the default threshold is a deliberate consequence of the class weighting, not a weakness of the models. Since the weighted loss corresponds to treating a missed failure as about 28.5 times more costly than a false alarm, the threshold should be chosen from the actual maintenance costs. Table IV gives the information needed for that choice, and the dashboard lets users explore it interactively.

The explanations add value beyond the metrics. The global ranking, the local attributions and the what-if curves are consistent with each other and with the physical rules of the dataset. This kind of evidence is what allows an engineer to accept an alarm or to spot a model that relies on spurious features.

The study has limitations. The AI4I 2020 data are synthetic, generated to reflect real milling conditions, so the absolute numbers may not transfer to a specific plant. Results come from a single stratified split and seed, and the test set contains only 51 failures, so small differences between models should be treated with caution. SHAP values can also be harder to interpret when inputs are strongly correlated, as the two temperatures are here.

VII. Conclusion

This paper presented an explainable deep learning pipeline for machine failure prediction on the AI4I 2020 dataset. Under identical training conditions with a class-weighted loss, an MLP achieved a ROC-AUC of 0.976, a PR-AUC of 0.720 and a recall of 90.2%, outperforming an LSTM and a 1D-CNN, while the 1D-CNN gave the highest recall of 94.1%. Threshold tuning raised the MLP F1-score to 0.724. Exact SHAP explanations identified the temperatures, tool wear and product type as the main drivers of risk, and a what-if analysis showed that the model's decision boundaries match the tool-wear and power-failure mechanisms of the process. With a median explanation latency below 20 ms, the system can explain every prediction in real time.

Future work will evaluate the approach on real sensor time series where temporal models can exploit history, use repeated cross-validation with confidence intervals, compare against gradient-boosted tree baselines, and predict the failure mode in addition to the failure itself.

References

[1] R. K. Mobley, An Introduction to Predictive Maintenance, 2nd ed. Oxford, U.K.: Butterworth-Heinemann, 2002.

[2] T. Zonta, C. A. da Costa, R. da Rosa Righi, M. J. de Lima, E. S. da Trindade, and G. P. Li, “Predictive maintenance in the Industry 4.0: A systematic literature review,” Computers & Industrial Engineering, vol. 150, Art. no. 106889, 2020.

[3] T. P. Carvalho, F. A. A. M. N. Soares, R. Vita, R. da P. Francisco, J. P. Basto, and S. G. S. Alcalá, “A systematic literature review of machine learning methods applied to predictive maintenance,” Computers & Industrial Engineering, vol. 137, Art. no. 106024, 2019.

[4] Y. Ran, X. Zhou, P. Lin, Y. Wen, and R. Deng, “A survey of predictive maintenance: Systems, purposes and approaches,” arXiv:1912.07383, 2019.

[5] R. Zhao, R. Yan, Z. Chen, K. Mao, P. Wang, and R. X. Gao, “Deep learning and its applications to machine health monitoring,” Mechanical Systems and Signal Processing, vol. 115, pp. 213–237, 2019.

[6] H. He and E. A. Garcia, “Learning from imbalanced data,” IEEE Trans. Knowl. Data Eng., vol. 21, no. 9, pp. 1263–1284, Sep. 2009.

[7] A. Adadi and M. Berrada, “Peeking inside the black-box: A survey on explainable artificial intelligence (XAI),” IEEE Access, vol. 6, pp. 52138–52160, 2018.

[8] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 30, 2017, pp. 4765–4774.

[9] S. Matzka, “Explainable artificial intelligence for predictive maintenance applications,” in Proc. 3rd Int. Conf. Artificial Intelligence for Industries (AI4I), 2020, pp. 69–74.

[10] S. Matzka, “AI4I 2020 predictive maintenance dataset,” UCI Machine Learning Repository, 2020. [Online]. Available: https://doi.org/10.24432/C5HS5C

[11] S. Kiranyaz, O. Avci, O. Abdeljaber, T. Ince, M. Gabbouj, and D. J. Inman, “1D convolutional neural networks and applications: A survey,” Mechanical Systems and Signal Processing, vol. 151, Art. no. 107398, 2021.

[12] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[13] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 1135–1144.

[14] L. S. Shapley, "A value for n-person games," in Contributions to the Theory of Games, vol. II, H. W. Kuhn and A. W. Tucker, Eds. Princeton, NJ, USA: Princeton Univ. Press, 1953, pp. 307–317.

[15] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," J. Mach. Learn. Res., vol. 15, pp. 1929–1958, 2014.

[16] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in Proc. 32nd Int. Conf. Machine Learning (ICML), 2015, pp. 448–456.

[17] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. 3rd Int. Conf. Learning Representations (ICLR), 2015.

[18] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 32, 2019, pp. 8024–8035.

[19] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[20] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," PLoS ONE, vol. 10, no. 3, Art. no. e0118432, 2015.

[21] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS) Datasets and Benchmarks Track, vol. 35, 2022.